Chapter 14 – Coding Best Practices

Objective

Assess whether the candidate understands how to write clean, readable, maintainable, and efficient code. The focus is not on syntax but on writing software that is easy to understand, debug, test, and maintain.

Q1. What do you mean by Coding Best Practices?

Difficulty: Basic

Expected Answer

Coding best practices are guidelines and techniques that help developers write code that is:

- Readable
- Maintainable
- Reusable
- Efficient
- Easy to debug
- Easy to test

The goal is to make code understandable for both the original developer and other team members.

Follow-up Questions

- Why are coding standards important?
- Who benefits from clean code?

Common Mistake

- Thinking best practices are only about formatting.

Red Flag

- Says, "If the code works, it's good enough."

Q2. Why is code readability important?

Difficulty: Basic

Expected Answer

Readable code:

- Is easier to understand.
- Reduces bugs.
- Simplifies maintenance.
- Helps new developers understand the project quickly.
- Improves collaboration.

Java Example

Bad

```
int a,b,c;
```

Good

```
int studentAge;
int totalMarks;
int attendanceCount;
```

Follow-up Questions

- Who reads code more often: humans or computers?
- Would you prefer short names or meaningful names?

Common Mistake

- Using one-letter variable names everywhere.

Red Flag

- Doesn't care about naming.

Q3. What makes code maintainable?

Difficulty: Basic

Expected Answer

Maintainable code should:

- Be modular.
- Use meaningful names.
- Avoid duplication.

- Be properly organized.
- Be easy to modify.
- Be well documented where necessary.

Follow-up Questions

- Why is duplicate code a problem?
- What happens if business requirements change?

Common Mistake

- Thinking comments alone make code maintainable.

Q4. What is the DRY Principle?

Difficulty: Intermediate

Expected Answer

DRY stands for **Don't Repeat Yourself**.

It means:

- Avoid duplicate code.
- Write reusable logic.
- Keep a single source of truth.

Benefits:

- Easier maintenance.
- Fewer bugs.
- Faster updates.

Follow-up Questions

- Can too much abstraction also become a problem?
- How would you remove duplicate code?

Common Mistake

- Copy-pasting code instead of creating reusable methods.

Q5. What is the KISS Principle?

Difficulty: Intermediate

Expected Answer

KISS stands for **Keep It Simple, Stupid**.

It means:

- Prefer simple solutions.
- Avoid unnecessary complexity.
- Write code that is easy to understand.

Simple code is easier to:

- Test
- Debug
- Maintain

Follow-up Questions

- Is the shortest code always the best?
- Can over-engineering create problems?

Common Mistake

- Trying to use advanced features unnecessarily.

Q6. Why should functions be small and focused?

Difficulty: Intermediate

Expected Answer

A good function should:

- Perform one responsibility.
- Be easy to understand.
- Be easy to test.
- Be reusable.
- Reduce complexity.

This follows the **Single Responsibility Principle (SRP)**.

Follow-up Questions

- How do you know when a function is too large?
- Would a 500-line function be acceptable?

Common Mistake

- Writing one function that performs many unrelated tasks.

Q7. Why should magic numbers and hardcoded values be avoided?

Difficulty: Intermediate

Expected Answer

Magic numbers make code difficult to understand.

Instead:

- Use named constants.
- Store configuration externally when appropriate.

Java Example

Bad

if(age > 18)

Better

private static final int MINIMUM_AGE = 18;

Follow-up Questions

- Why are constants useful?
- Where should configuration values be stored?

Common Mistake

- Hardcoding business rules throughout the codebase.

Q8. Scenario-Based Question

Difficulty: Advanced

You receive a class containing:

- 2,000 lines of code.
- 50 methods.
- Poor variable names.
- Duplicate logic.

How would you improve it?

Expected Answer

Candidate should discuss:

- Break into smaller classes.
- Improve naming.
- Remove duplicate code.
- Extract reusable methods.
- Write unit tests before major refactoring.
- Refactor gradually.

Follow-up Questions

- Would you rewrite everything immediately?
- How would you ensure existing functionality isn't broken?

Common Mistake

- Suggesting a complete rewrite without understanding the risks.

Q9. Scenario-Based Question

Difficulty: Advanced

Your code works perfectly but another developer cannot understand it.

Would you consider it good code?

Expected Answer

No.

Good code should be:

- Correct.
- Readable.
- Maintainable.
- Understandable by other developers.
- Easy to modify.

The candidate should recognize that software is maintained by teams, not just individuals.

Follow-up Questions

- Who spends more time reading code than writing it?

- Why is maintainability important?

Q10. Real Interview Scenario

Difficulty: Advanced

Your manager asks you to deliver a feature quickly.

You have two options:

Option A

- Fast implementation.
- Poor coding practices.

Option B

- Slightly more time.
- Clean, maintainable code.

What would you choose and why?

Expected Answer

A strong candidate should discuss:

- Balancing deadlines with quality.
- Avoiding technical debt.
- Delivering maintainable code.
- Communicating trade-offs with stakeholders.
- Writing code that can be safely extended in the future.

The best answer acknowledges that business deadlines matter, but quality should not be sacrificed to the point where it creates long-term maintenance problems.

Follow-up Questions

- What is technical debt?
- Have you ever had to refactor code after a rushed release?

Common Mistake

- Focusing only on speed or only on perfection without considering business needs.

Red Flag

- Believes maintainability doesn't matter as long as the feature works.